LAPORAN JOBSHEET
“11- Dynamic Routing & Static Generation”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

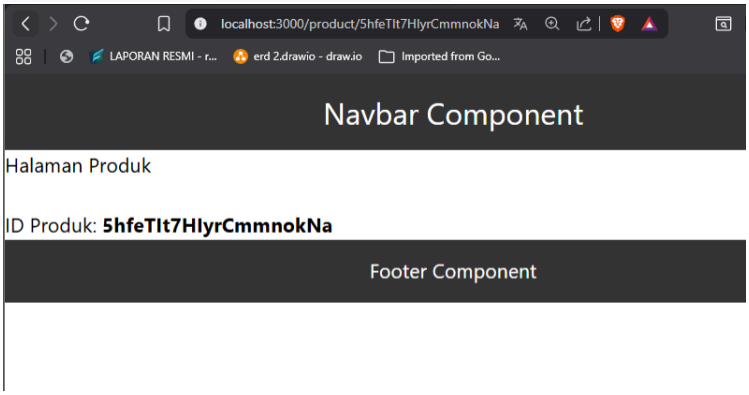
2026

Bagian 1 – Membuat Dynamic Route

1. Buka file views/product/index.tsx dan modfikasi

```
<Link
  href={`/product/${product.id}`} className={styles.product_button}>
  Beli Sekarang
</Link>
```

2. Jalankan browser http://localhost:3000/product
   - Ketika button Beli sekarang dipencet maka akan menavigasi ke halaman lain

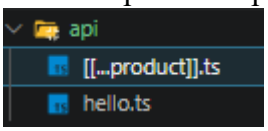


Bagian 2 – Implementasi CSR (Client Rendering)

1. Modifikasi pada file [produk].tsx pada folder src/pages/product/

```
index.tsx 2, M    [product].tsx 3, U
P2 > Praktikum > next-routing > src > pages > product > [product].tsx > ...
import { useRouter } from "next/router";
import fetcher from "@/utils/swr/fetcher";
import useSWR from "swr";

const HalamanProduk = () => {
//   const router = useRouter();
//   const { product } = router.query;
const {query: { product }} = useRouter();
const { data, error, isLoading } = useSWR( `/api/products/${product}`, fetcher);

  return (
    <div>
      <h1>Halaman Produk</h1>
      <br />
      <p>
        ID Produk: <strong>{product}</strong>
      </p>
    </div>
  );
};

export default HalamanProduk;
```

2. Pada file produk.ts pada folder pages/api di rename menjadi [[...product]].ts



3. Modifikasi file servicefirebase.ts

P2 > Praktikum > next-routing > src > utils > db > servicesfirebase.ts > retrieveDataById

```typescript
import { getFirestore, collection, getDocs, getDoc, doc } from "firebase/firestore";
import app from "./firebase";

const db = getFirestore(app);

export async function retrieveProducts<T>(
  collectionName: string,
): Promise<T[]> {
  try {
    console.log(`Fetching from collection: ${collectionName}`);
    const snapshot = await getDocs(collection(db, collectionName));

    console.log(`Total documents found: ${snapshot.docs.length}`);

    const data = snapshot.docs.map((doc) => {
      console.log(`Document ${doc.id}:`, doc.data());
      return {
        id: doc.id,
        ...doc.data(),
      };
    }) as T[];

    return data;
  } catch (error) {
    console.error("Error fetching products:", error);
    throw error;
  }
}

export async function retrieveDataById(collectionName: string, id: string) {
  const snapshot = await getDoc(doc(db, collectionName, id));
  const data = snapshot.data();
  return data;
}
```

4. Modifikasi file [[...produk]].ts

P2 > Praktikum > next-routing > src > pages > api > [[...product]].ts > handler

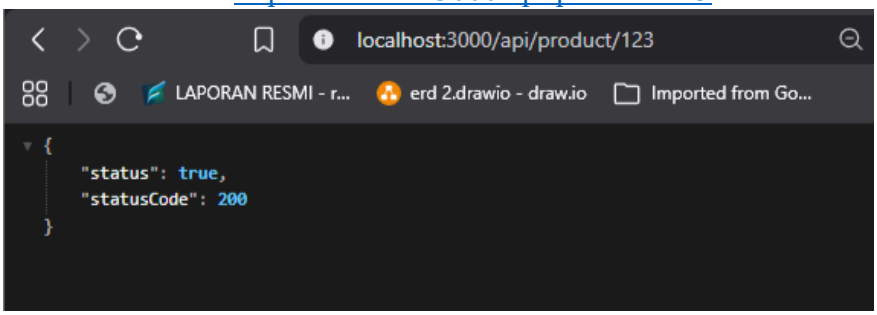
```typescript
import type { NextApiRequest, NextApiResponse } from "next";
import { retrieveProducts , retrieveDataById} from "../../utils/db/servicesfirebase";

type Data = {
  status: boolean;
  statusCode: number;
  /* eslint-disable @typescript-eslint/no-explicit-any */
  data: any;
};

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>,
) {
  if (req.query.product![1]) {
    const productData = await retrieveDataById("products", req.query.product![1]);
    res.status(200).json({ status: true, statusCode: 200, data: productData });
  } else {
    const data = await retrieveProducts("products");
    res.status(200).json({ status: true, statusCode: 200, data });
  }
}
```

5. Jalankan browser http://localhost:3000/api/product/5hfeTIt7HIyrCmmnokNa

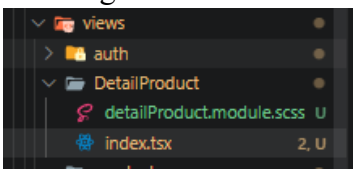
Raw


```
{
  "status": true,
  "statusCode": 200,
  "data": {
    "color": "black",
    "image": "https://cdn.shopify.com/s/files/1/1202/6102/products/nike-sb-seasonal-skate-jacket-black-black-black-white-1.jpg",
    "size": "XL",
    "price": 150000,
    "category": "jacket",
    "name": "Nike Crewneck "
  }
}
```

6. Jalankan browser http://localhost:3000/api/product/123



7. Buat file dengan nama index.tsx pada folder views/DetailProduct selain itu buat juga file dengan nama detailProduct.module.scss



8. Modifikasi detailProduct.module.scss

P2 > Praktikum > next-routing > src > views > DetailProduct > detailProduct.module.scss > .skeleton_title

```scss
.container {
  min-height: 100vh;
  background: linear-gradient(to bottom right, #fafafa, #f3f4f6);
  padding: 2rem 1rem;

  @media (min-width: 640px) {
    padding: 2rem 1.5rem;
  }

  @media (min-width: 1024px) {
    padding: 2rem 2rem;
  }
}

.maxWidth {
  max-width: 80rem;
  margin: 0 auto;
}

.backButton {
  display: inline-block;
  margin-bottom: 2rem;
  padding: 0.75rem 1.5rem;
  background-color: #f3f4f6;
  color: #4b5563;
  text-decoration: none;
  border-radius: 0.5rem;
  font-weight: 500;
  transition: all 0.2s;
  border: 1px solid #e5e7eb;

  &:hover {
    background-color: #e5e7eb;
    color: #1f2937;
  }
}

// Main Detail Wrapper
.detailWrapper {
  display: grid;
  grid-template-columns: 1fr;
  gap: 2rem;
  background-color: white;
  border-radius: 0.75rem;
  overflow: hidden;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);

  @media (min-width: 768px) {
    grid-template-columns: 1fr 1fr;
    gap: 3rem;
    padding: 2rem;
  }

  @media (min-width: 1024px) {
    grid-template-columns: 0.8fr 1.2fr;
    gap: 4rem;
    padding: 3rem;
  }
```

9. Modifikasi index.tsx pada folder DetailProduct

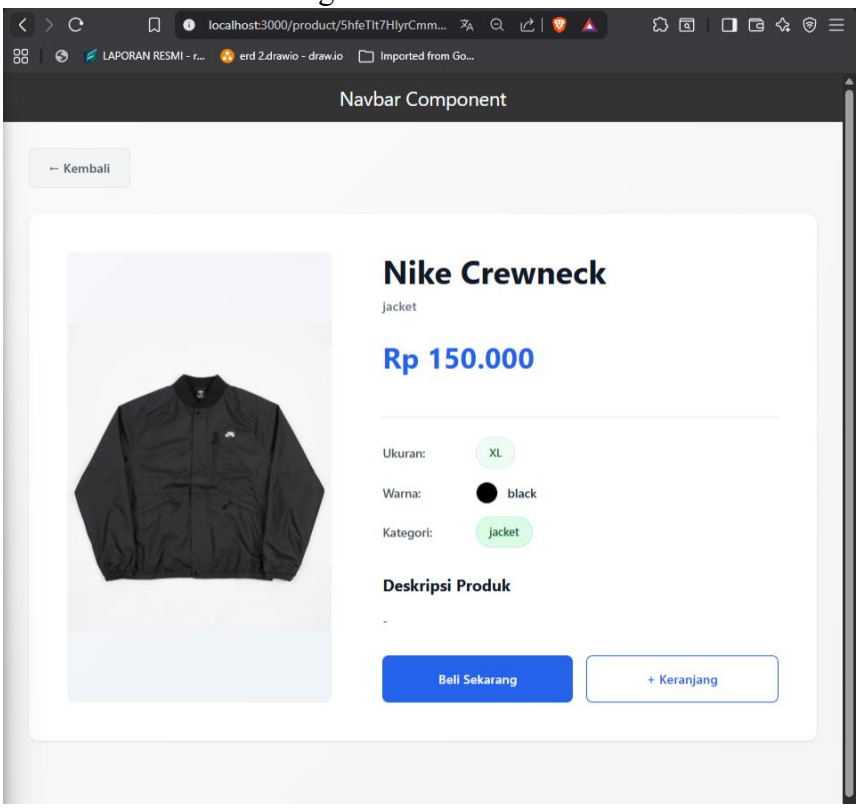
```tsx
import React from "react";
import styles from "./detailProduct.module.scss";
import { ProductType } from "../../types/product.type";
import Link from "next/link";

interface DetailProductProps {
  product?:
    | {
        status?: boolean;
        statusCode?: number;
        data?: ProductType;
      }
    | ProductType;
  isLoading?: boolean;
}

const DetailProduct: React.FC<DetailProductProps> = ({
  product,
  isLoading = false,
}) => {
  // Extract actual product data from API response structure
  let productData: ProductType | undefined;

  if (product && "data" in product) {
    productData = product.data;
  } else {
    productData = product as ProductType | undefined;
  }

  if (isLoading) {
    return (
      <div className={styles.container}>
        <div className={styles.maxWidth}>
          <Link href="/product" className={styles.backButton}>
            ← Kembali
          </Link>
          <div className={styles.detailWrapper}>
            <div className={styles.detailImage_skeleton}></div>
            <div className={styles.detailContent}>
              <div className={styles.skeleton_title}></div>
              <div className={styles.skeleton_subtitle}></div>
              <div className={styles.skeleton_price}></div>
              <div className={styles.detailSection}>
                <div className={styles.skeleton_text}></div>
                <div className={styles.skeleton_text}></div>
              </div>
            </div>
          </div>
```

10. Modifikasi file [product].tsx

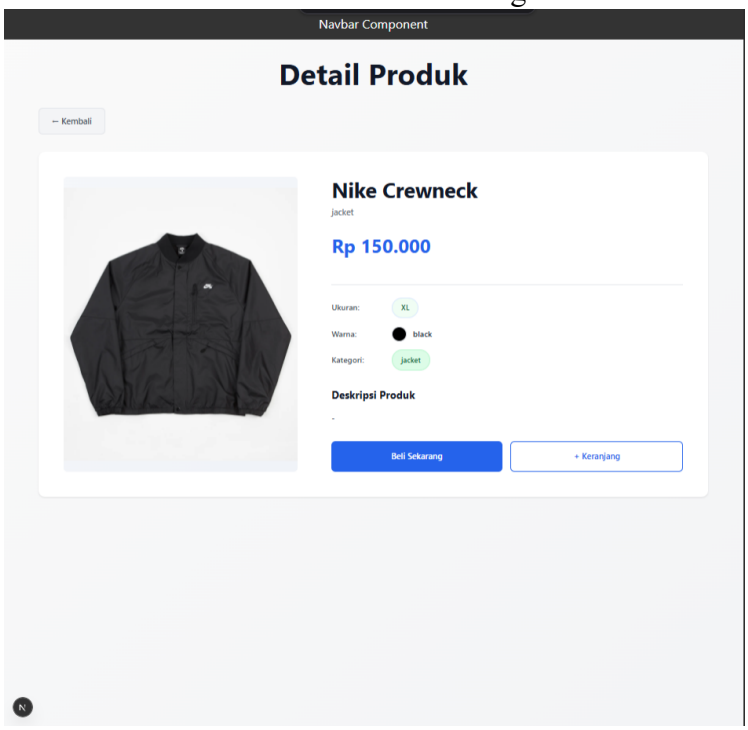
```tsx
import { useRouter } from "next/router";
import fetcher from "@/utils/swr/fetcher";
import useSWR from "swr";
import DetailProduct from "@/views/DetailProduct";

const HalamanProduk = () => {
//    const router = useRouter();
//    const { product } = router.query;
const {query: { product }} = useRouter();
const { data, error, isLoading } = useSWR( `/api/products/${product}`, fetche

  return (
    <div>
      <DetailProduct product={data} isLoading={isLoading} />
    </div>
  );
};

export default HalamanProduk;
```

11. Jalankan browser http://localhost:3000/product/ saat produk diklik maka akan muncul detailProduct http://localhost:3000/product/5hfeTIt7HIyrCmmnokNa

Ketika klik Beli Sekarang maka akan membuka



12. Membuat Tulisan Detail Product di Tengah



Bagian 3 – Implementasi SSR

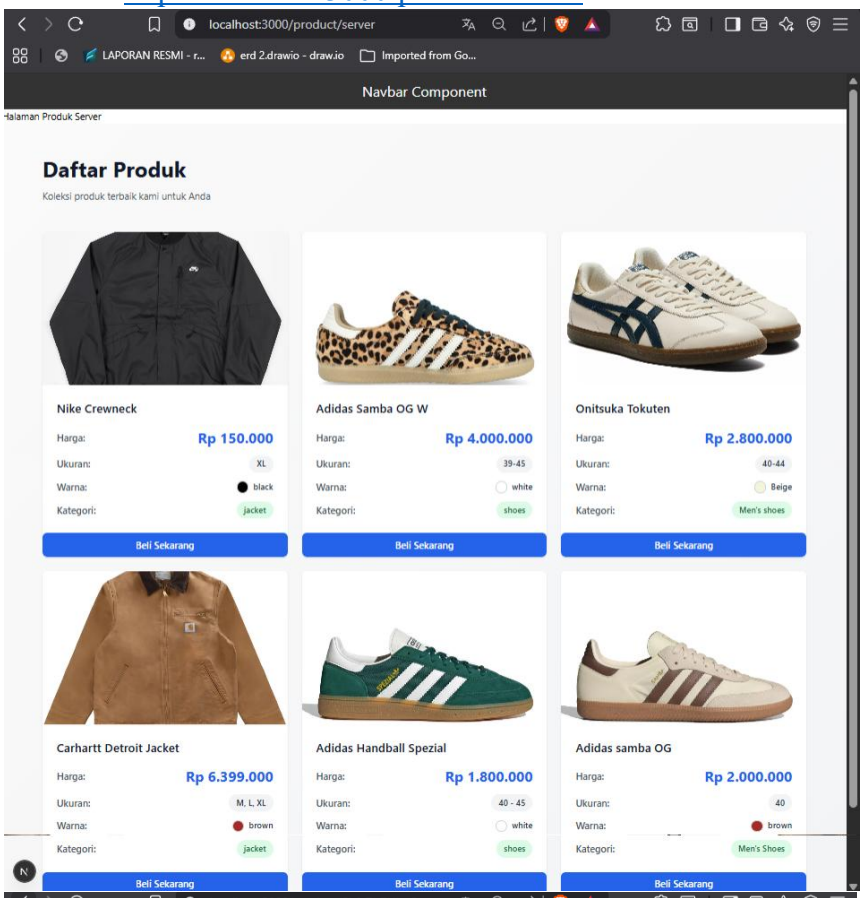
1. Modifikasi [produk].tsx pada folder src/pages/produk dan comment line 9 sampai 20 dikarena kita akan menggunakan metode SSR. Tambahkan beberapa kode untuk SSR

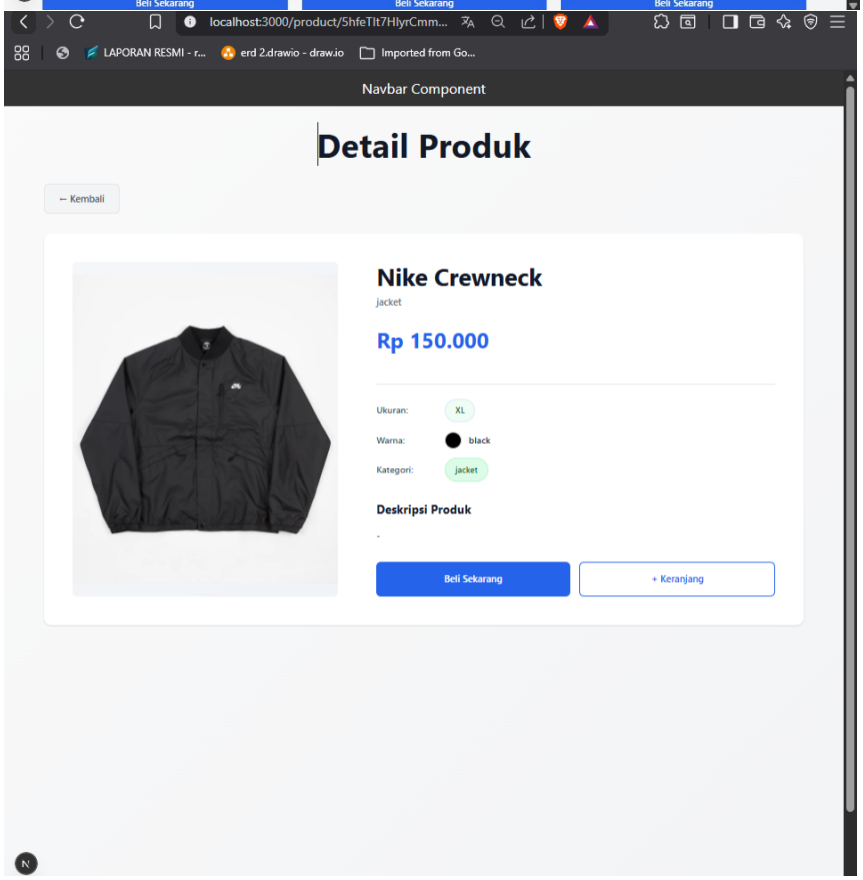
index.tsx 2, U × | detailProduct.module.scss U | [product].tsx 3, U ×

P2 > Praktikum > next-routing > src > pages > product > [product].tsx > getServerSideProps

```
4   import DetailProduct from "@/views/DetailProduct";
5   import { ProductType } from "@/types/product.type";
6
7   const HalamanProduk = ({ product }: { product: ProductType }) => {
8   // (digunakan untuk client-side rendering)
9   //   const router = useRouter();
10  //   const { product } = router.query;
11  // const {query: { product }} = useRouter();
12  // const { data, error, isLoading } = useSWR( `/api/products/${product}`, fetcher);
13
14  //   return (
15  //     <div>
16  //       <DetailProduct product={data} isLoading={isLoading} />
17  //     </div>
18  //   );
19  // };
20
21    return (
22      <div>
23        <DetailProduct product={product}/>
24      </div>
25    );
26  }
27
28  export default HalamanProduk;
29
30  // (untuk server-side rendering)
31  export async function getServerSideProps({params}:{ params : {product: string}}) {
32    const res = await fetch (`http://localhost:3000/api/products/${params?.product}`);
33    const response = await res.json();
34
35    return {
36      props: {
37        product: response.data,
38      },
39    };
40  }
```

2. Jalankan http://localhost:3000/product/server





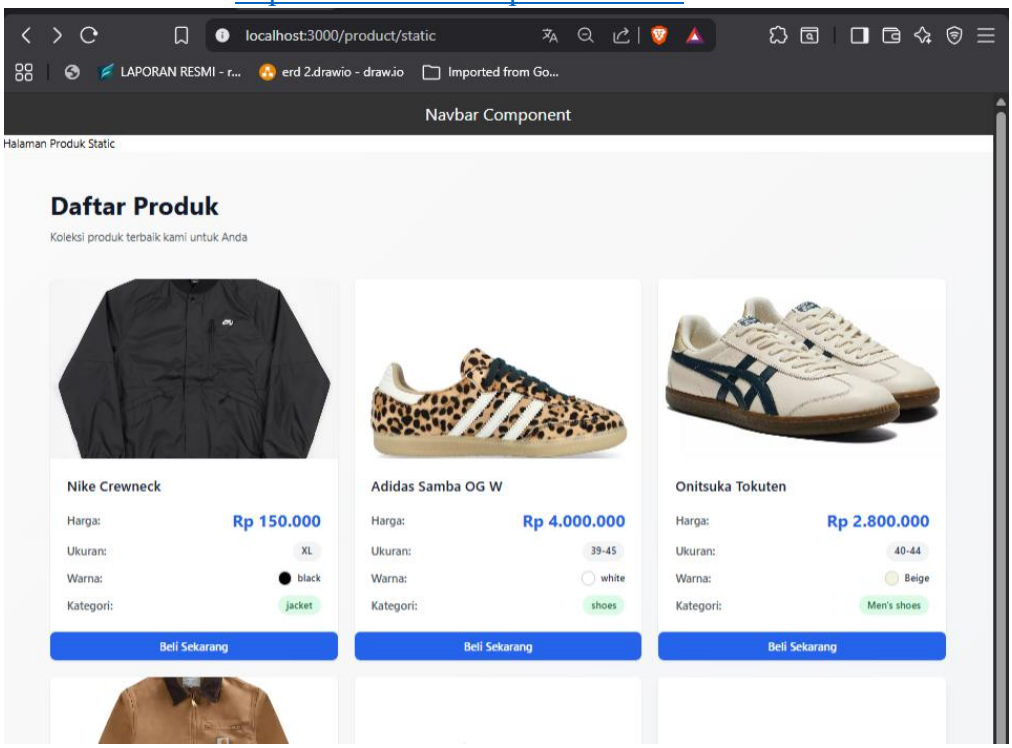
Tidak perlu loading state karena data sudah tersedia sebelum render

Bagian 4 – Implementasi Static Site Generation (Dynamic SSG

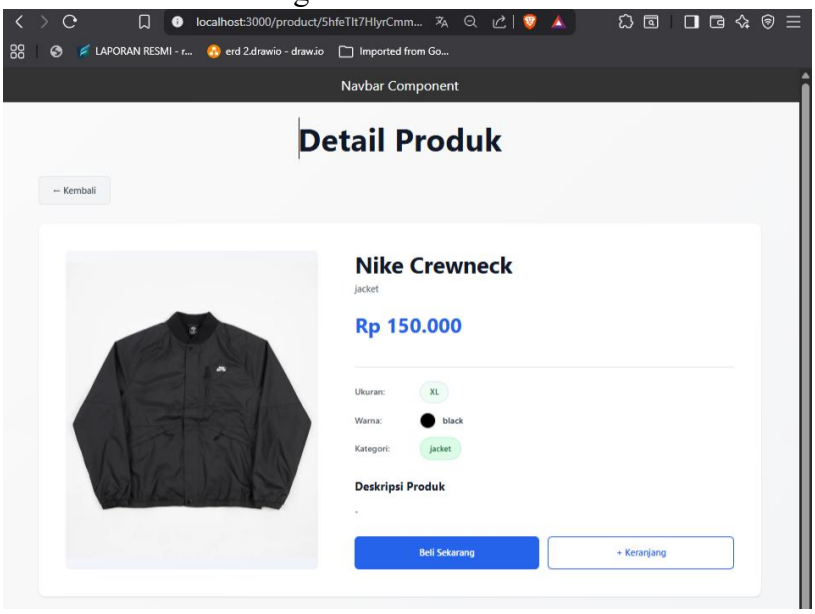
1. Buka file [produk].tsx dan modifikasi

```
P2 > Praktikum > next-routing > src > pages > product > [product].tsx > ...
import { useRouter } from "next/router";
import fetcher from "@/utils/swr/fetcher";
import useSWR from "swr";
import DetailProduct from "@/views/DetailProduct";
import { ProductType } from "@/types/product.type";

const HalamanProduk = ({ product }: { product: ProductType }) => {
  // (digunakan untuk client-side rendering)···

  return (
    <div>
      <DetailProduct product={product} />
    </div>
  );
};

export default HalamanProduk;

// (untuk server-side rendering)···
// (untuk static site generation)
export async function getStaticPaths() {
  const res = await fetch(`http://localhost:3000/api/products`);
  const response = await res.json();

  const paths = response.data.map((product: ProductType) => ({
    params: { product: product.id.toString() },
  }));
  return { paths, fallback: false };
}

export async function getStaticProps({
  params,
}: {
  params: { product: string };
}) {
  const res = await fetch(
    `http://localhost:3000/api/products/${params?.product}`,
  );
  const response: { data: ProductType[] } = await res.json();

  return {
    props: {
      product: response.data[0],
    },
  };
}
```

2. Jalankan browser http://localhost:3000/product/static



Ketika di Beli Sekarang di klik



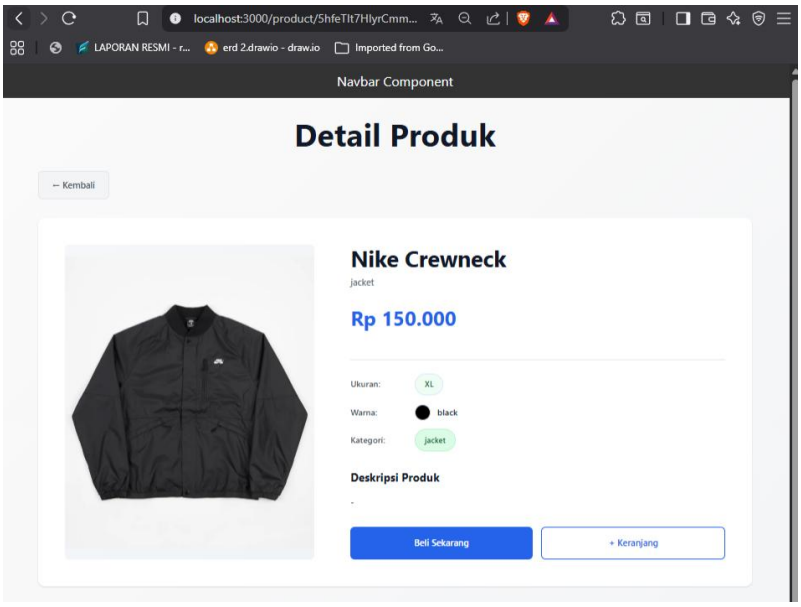
PENGUJIAN

1. Uji 1 – CSR

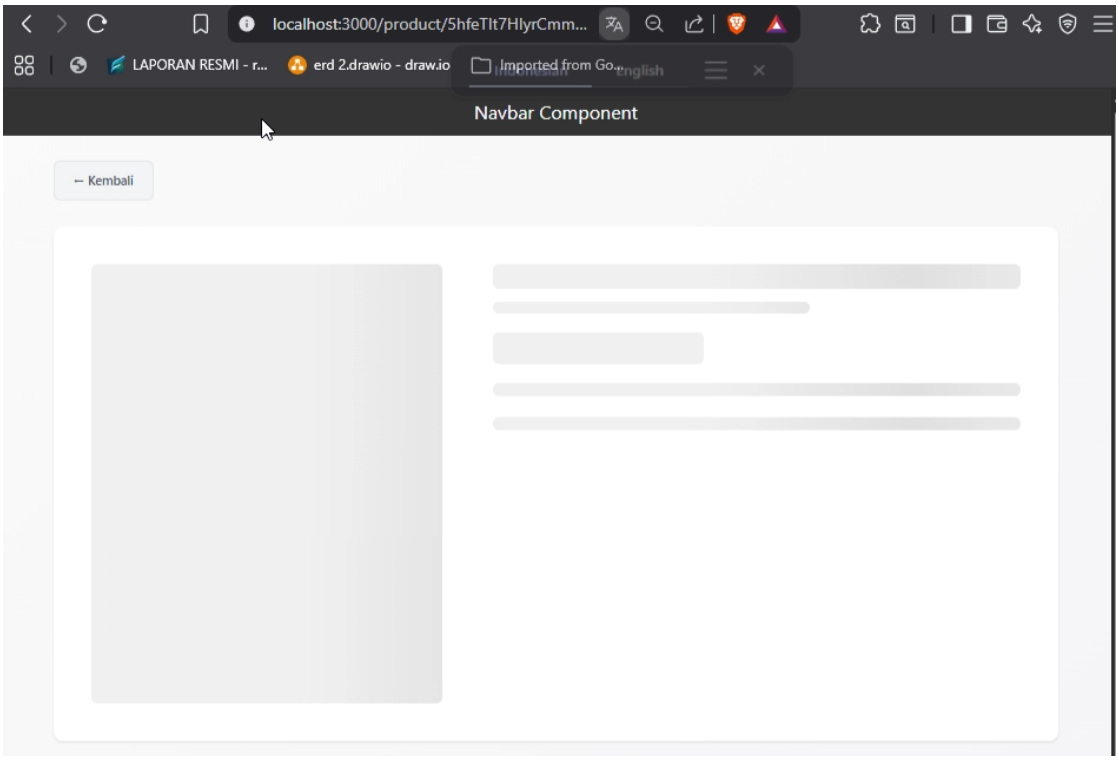
```
const HalamanProduk = () => {
  // (digunakan untuk client-side rendering)
  //   const router = useRouter();
  //   const { product } = router.query;
  const {
    query: { product },
  } = useRouter();
  const { data, error, isLoading } = useSWR(
    `/api/products/${product}`,
    fetcher,
  );

  return (
    <div>
      <DetailProduct product={data} isLoading={isLoading} />
    </div>
  );
```

- Refresh halaman detail

  Awal

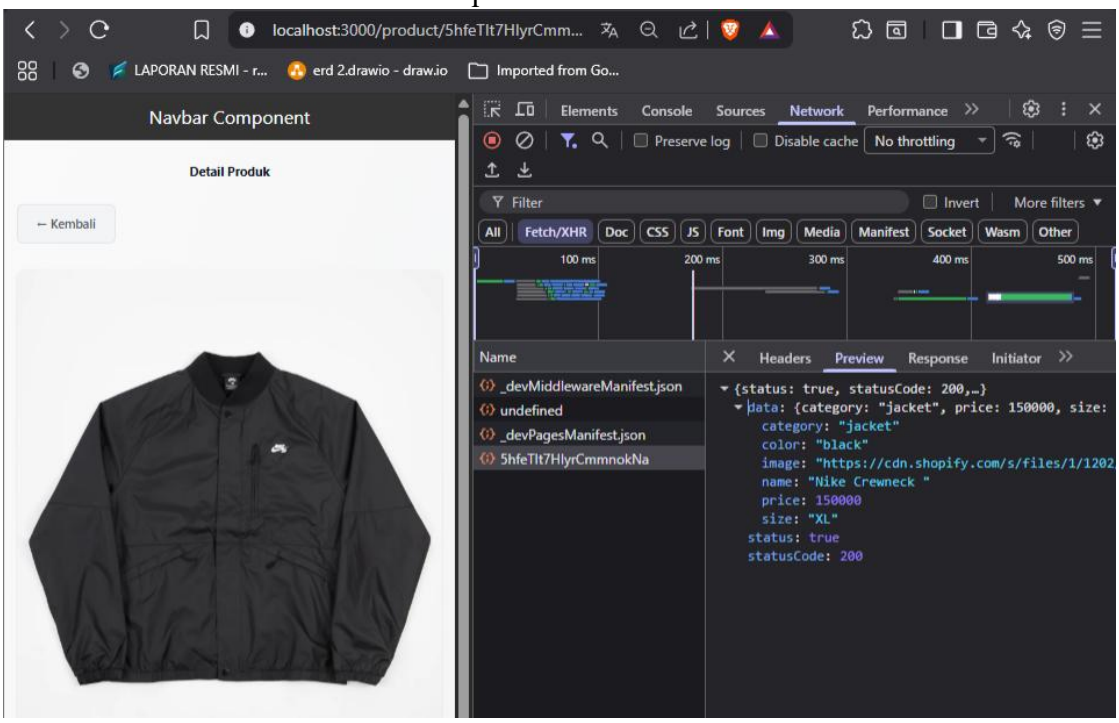


Setelah di refresh



Terdapat skeleton loading ketika di refresh

- Periksa Network → XHR → API request terlihat

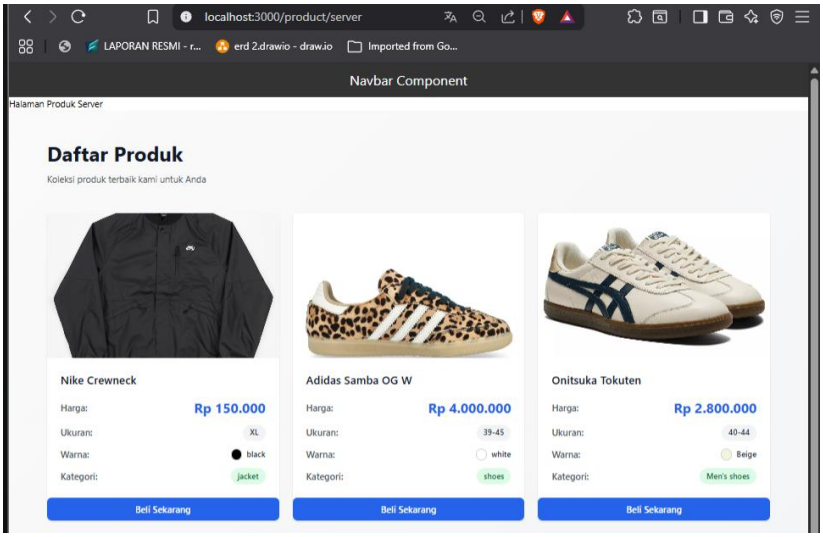


2. Uji 2 – SSR

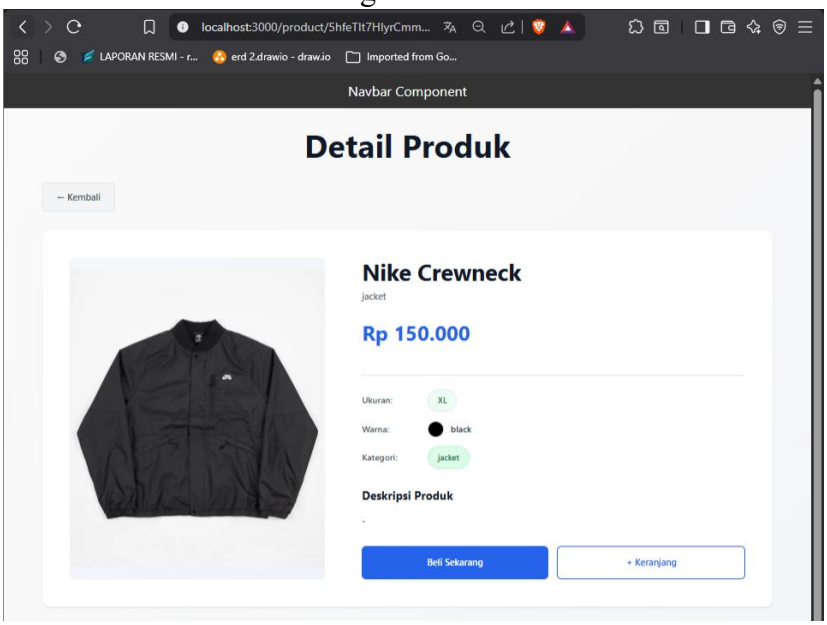
```
7   const HalamanProduk = ({ product }: { product: ProductType }) => {
8 > //   // (digunakan untuk client-side rendering) ···
20
21    return (
22      <div>
23        <DetailProduct product={product} />
24      </div>
25    );
26  };
27
28  export default HalamanProduk;
29
30  // (untuk server-side rendering)
31  export async function getServerSideProps({params}:{ params : {product: string}}) {
32    const res = await fetch (`http://localhost:3000/api/products/${params?.product}`);
33    const response = await res.json();
34
35    return {
36      props: {
37        product: response.data,
38      },
39    };
40  }
41
```

- Refresh halaman detail

  Awal



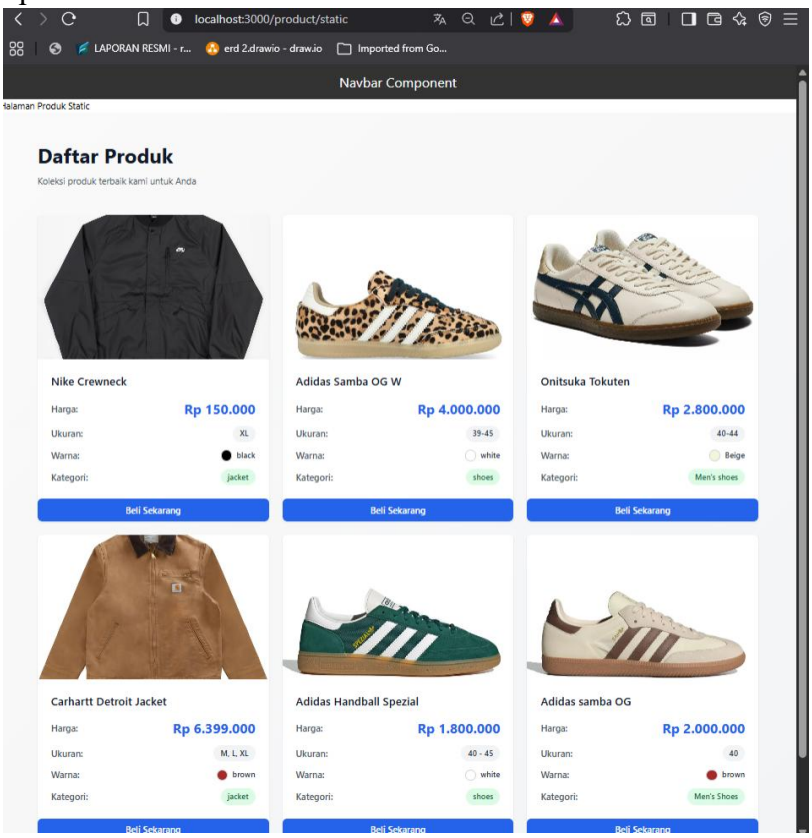
  Setelah di klik Beli Sekarang



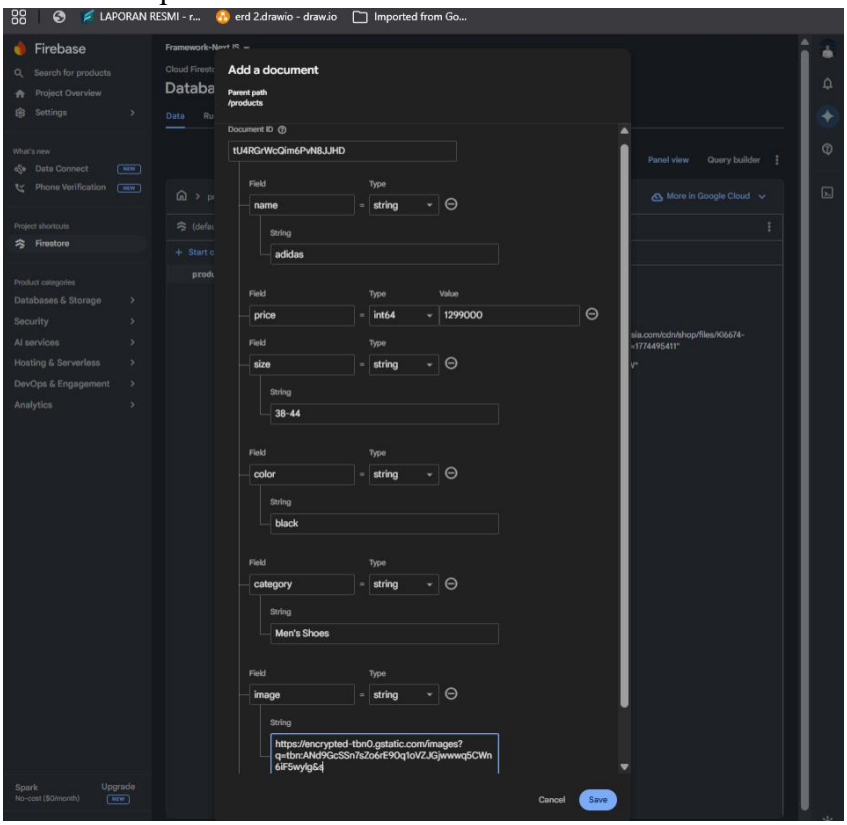
  Setelah di refresh akan tetap menampilkan tanpa adanya skeleton loading

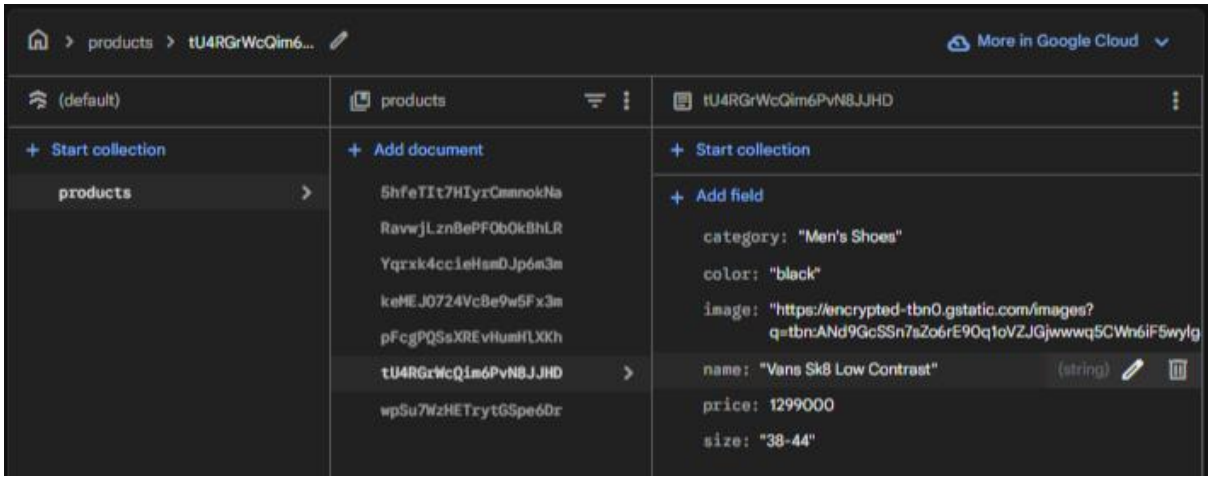
Uji 3 – SSG (Lakukan seperti langkah sebelumya pada Jobsheet 10)

1. Jalankan:
   - npm run build
   - npm run start

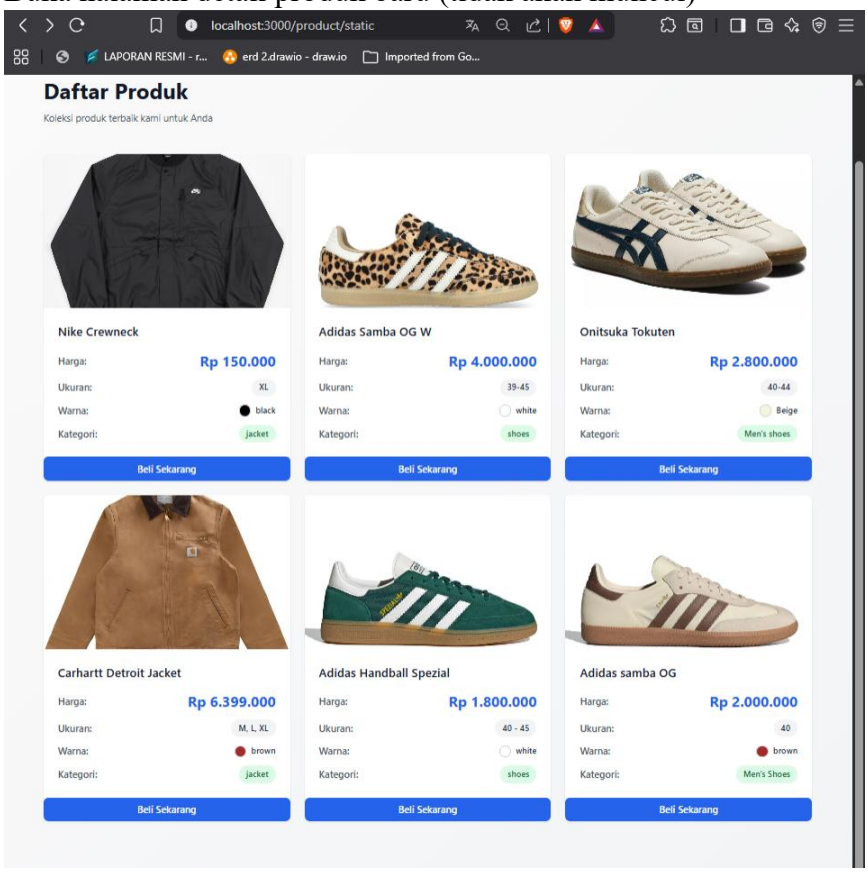


2. Tambahkan produk baru di database.



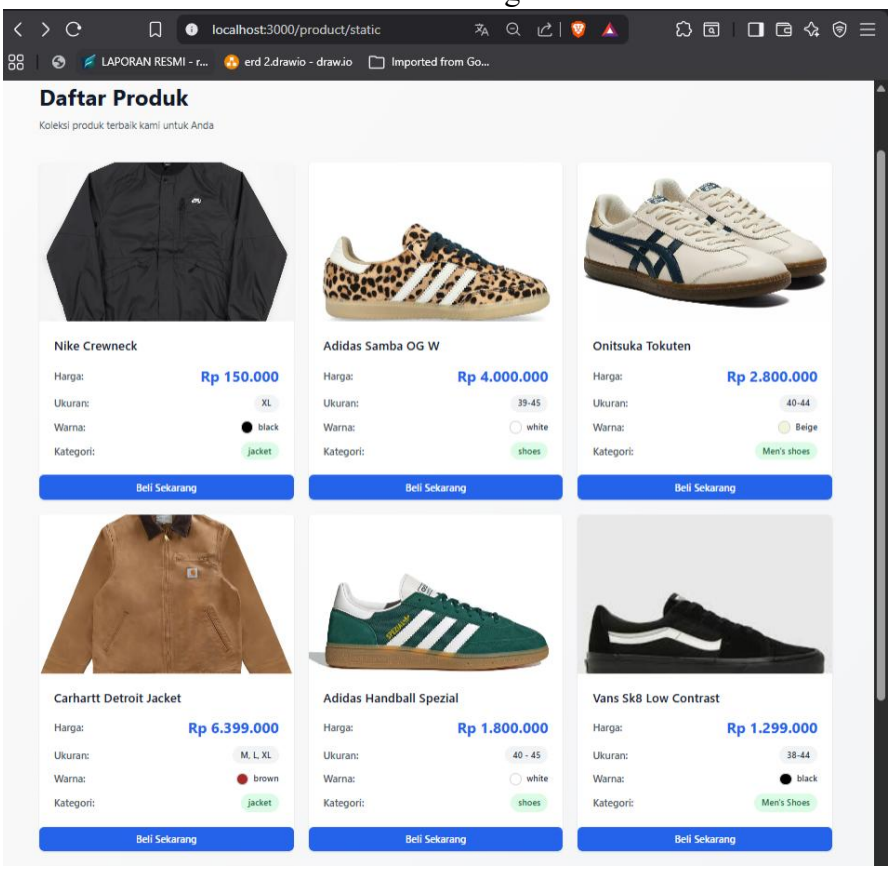


3. Buka halaman detail produk baru (tidak akan muncul)

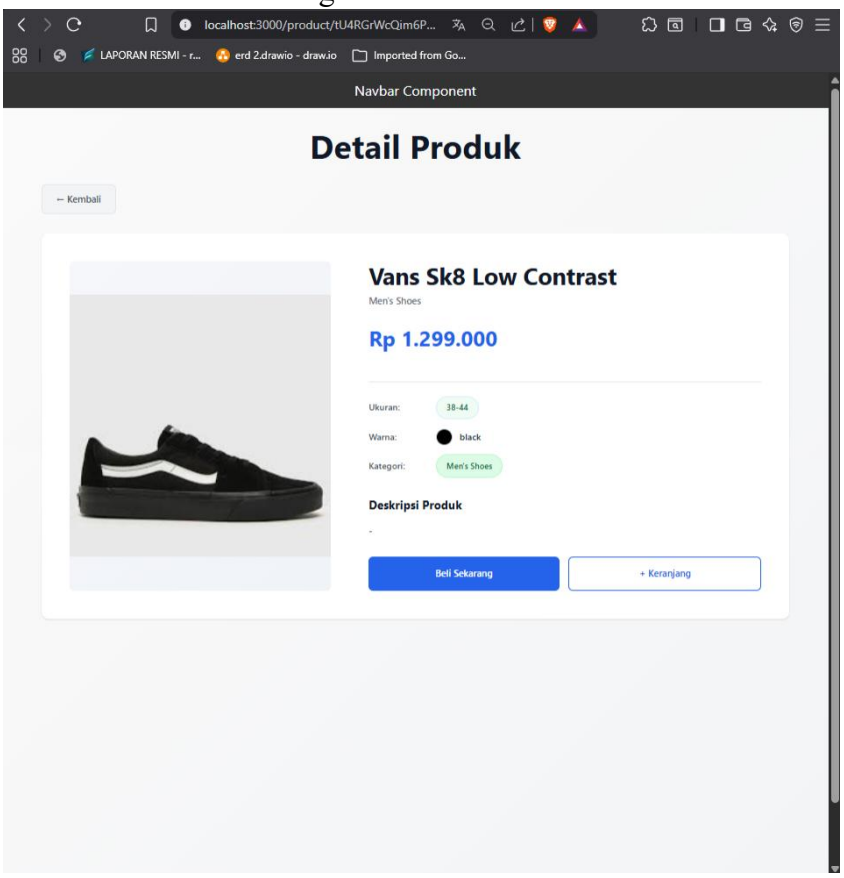


4. Build ulang:
   - npm run build
   - npm run star

Baru akan muncul setelah di build ulang



Ketika klik Beli Sekarang



TUGAS PRAKTIKUM

1. Implementasikan halaman detail dengan:
   - CSR
     Pada metode CSR, data diambil dari API di sisi client setelah halaman dirender. Saat halaman detail dibuka atau di-refresh, akan muncul loading/skeleton terlebih dahulu karena data belum tersedia. Setelah request API selesai, barulah data ditampilkan.

   - SSR
     Pada SSR, data diambil di server sebelum halaman dikirim ke browser. Hasilnya, saat halaman dibuka atau di-refresh, data langsung tampil tanpa loading, karena sudah diproses di server terlebih dahulu.

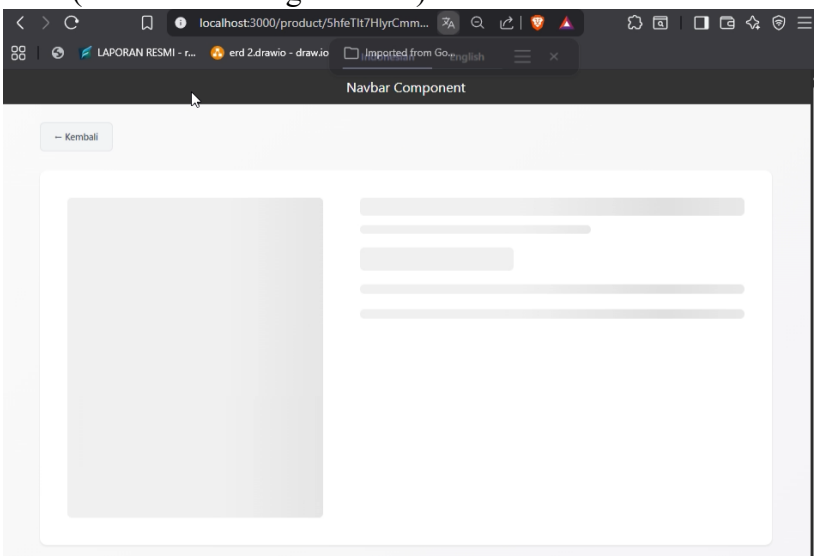
   - SSG
     Pada SSG, halaman dibuat saat proses build (npm run build). Data sudah menjadi file statis. Namun, jika ada data baru di database, halaman tidak akan langsung berubah dan harus build ulang agar data baru muncul.

2. Buat tabel perbandingan:

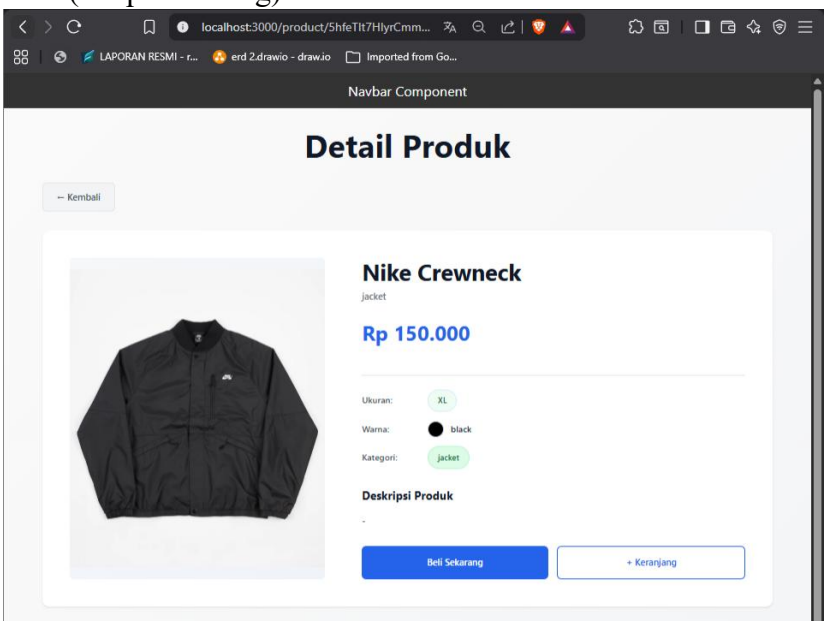
Aspek	CSR	SSR	SSG
Loading	Ada Loading Skeleton	Tidak ada Loading	Tidak ada Loading
Build Required	Tidak perlu rebuild	Tidak perlu rebuild	Perlu build ulang jika data bertambah/berubah
SEO (Search Engine Optimization)	SEO kurang bagus karena data baru muncul setelah halaman dimuat	SEO lebih baik karena konten sudah tersedia saat halaman dibuka	SEO lebih baik karena konten sudah tersedia saat halaman dibuka
Perubahan Data	Real-time (malah langsung update)	Real-time	Tidak real-time (harus build ulang)

3. Dokumentasikan:
   - **Screenshot**

CSR (memiliki loading skeleton)



SSR (Tanpa Loading)



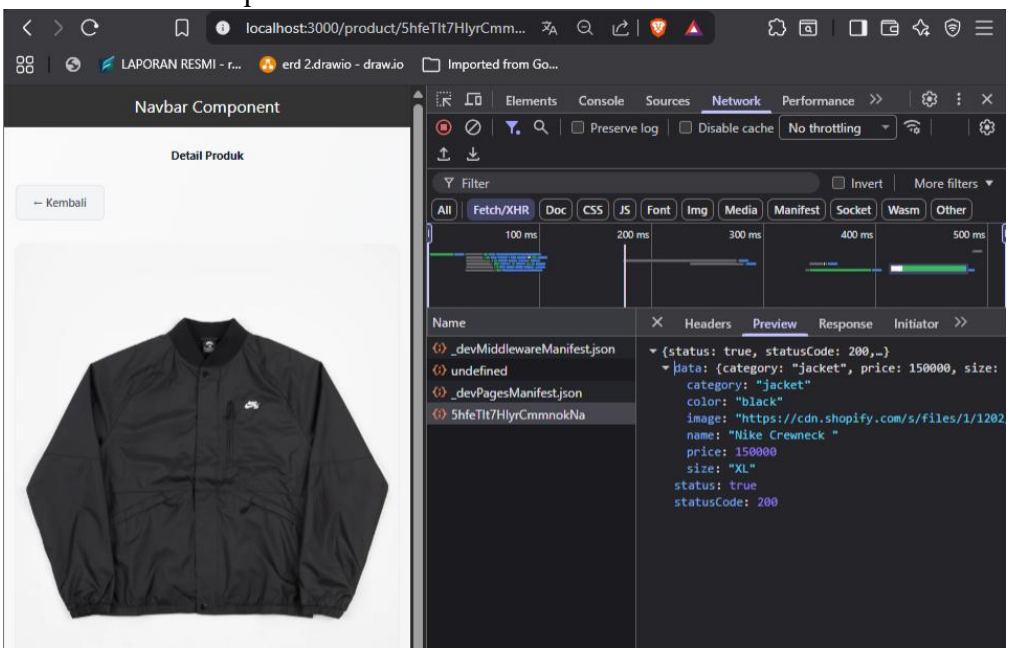
SSG (Harus Build ulang)

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm run build

Route (pages)
┌ ○ /
├   /_app
├ ○ /404
├ ○ /about
├ ƒ /api/[[...product]]
├ ƒ /api/hello
├ ○ /auth/login
├ ○ /auth/register
├ ○ /blog
├ ○ /blog/[slug]
├ ○ /category/[[...slug]]
├ ○ /product
├ ● /product/[product] (904 ms)
│ ├ /product/5hfeTIt7HIyrCmmnokNa
│ ├ /product/RavwjLznBePFObOkBhLR
│ ├ /product/Yqrxk4ccieHsmDJp6m3m
│ └ [+4 more paths]
├ ƒ /product/server
├ ● /product/static
├ ○ /profile
├ ○ /profile/edit
├ ○ /setting/app
├ ○ /shop/[[...slug]]
├ ○ /user
└ ○ /user/password

○  (Static)   prerendered as static content
●  (SSG)      prerendered as static HTML (uses getStaticProps)
ƒ  (Dynamic)  server-rendered on demand
```

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm run start

> next-routing@0.1.0 start
> next start

▲ Next.js 16.1.6
- Local:        http://localhost:3000
- Network:      http://100.102.141.86:3000
```

- **Network tab**

  CSR → terlihat request API di tab XHR/Fetch



  SSR → tidak terlihat request dari browser (karena di server)

  SSG → tidak ada request (data sudah statis)

- **Build result**

  Saat npm run build maka SSG akan generate halaman statis

  Produk baru tidak muncul sebelum build ulang

  Setelah build ulang maka produk baru muncul

PERTANYAAN ANALISIS

1. Mengapa getStaticPaths wajib pada dynamic SSG?
   Karena pada SSG, halaman dibuat saat build time, jadi Next.js harus tahu path mana saja yang perlu dibuat. getStaticPaths digunakan untuk mendefinisikan daftar route dinamis (misalnya id produk), tanpa ini Next.js tidak tahu halaman mana yang harus di-generate.

2. Mengapa CSR membutuhkan loading state?
   Karena pada CSR data diambil setelah halaman dirender di browser, sehingga ada jeda waktu saat menunggu response API. Loading state diperlukan untuk memberi feedback ke user agar tidak terlihat kosong atau error.

3. Mengapa SSG tidak menampilkan produk baru tanpa build ulang?
   Karena SSG menghasilkan halaman statis saat build, jadi data yang digunakan adalah snapshot lama. Produk baru di database tidak akan muncul sampai dilakukan build ulang agar halaman diperbarui.

4. Mana metode terbaik untuk halaman detail e-commerce?
   Umumnya SSR adalah pilihan terbaik, karena:
   - Data selalu up-to-date secara real-time
   - Tidak perlu loading seperti CSR
   - SEO tetap bagus

   Namun, jika data jarang berubah, SSG bisa digunakan untuk performa lebih cepat.

5. Apa risiko menggunakan SSG untuk produk yang sering berubah?
   - Data bisa tidak akurat (stale)
   - Harga/stok tidak update real-time
   - User bisa melihat informasi lama
   - Harus sering build ulang (tidak efisien)